Table of Contents

1. *Project Overview …… 02*
2. *Project Objectives …… 02*
3. *Code Explanation …… 02*
    - *Filter Logic …… 02*
    - *Protocol Detection …… 02*
    - *Payload Decoding …… 02*
4. *Key Features …… 03*
5. *How to Run …… 03*
6. *What I Learned …… 03*
7. *Conclusion …… 04*

Python Network Packet Sniffer

1. Project Overview

*The **Python Network Packet Sniffer** is a specialized tool designed to intercept, inspect, and analyse data packets as they travel across a network interface. Built using the **Scapy** framework, this project serves as a functional demonstration of how data encapsulation works within the OSI model. It provides real-time visibility into the "conversations" happening between devices on a local area network (LAN).*

2. Project Objectives

- ***Packet Interception:** Capture live traffic from the network interface card (NIC).*
- ***Protocol Identification:** Automate the detection of Layer 3 (IP) and Layer 4 (TCP/UDP) protocols.*
- ***Data Extraction:** Isolate and display source/destination metadata and raw payload content.*
- ***Security Research:** Provide a foundation for identifying unencrypted traffic and potential security vulnerabilities.*

3. Code Explanation

The script is structured into three primary logical components:

A. The Filter Logic (if packet.haslayer(IP):)

*The sniffer filters out lower-level noise (like ARP or Ethernet frames) to focus on the **Internet Protocol (IP)** layer. This allows the tool to extract the "Who" and "Where" of the packet via src and dst attributes.*

B. Protocol Detection

The script uses conditional logic to probe the packet structure:

- ***TCP:** Identifies connection-oriented traffic (e.g., Web browsing, SSH).*
- ***UDP:** Identifies connectionless traffic (e.g., DNS, Streaming).*

C. Payload Decoding

The packet.payload section attempts to convert the raw data into a readable byte format. A try-except block is implemented here as a safety measure to ensure the script doesn't crash when encountering non-standard or encrypted binary data.

4. Key Features

- ***Non-Persistent Storage:*** *Uses store=False to ensure the script can run for hours without consuming the system's RAM.*
- ***Real-Time Output:*** *Immediate console feedback as packets hit the interface.*
- ***Lightweight Footprint:*** *Requires minimal code to achieve deep packet inspection (DPI).*
- ***Cross-Protocol Support:*** *Handles the most common protocols used in modern networking.*

5. How to Run

Prerequisites

- ***Python 3.x*** *installed.*
- ***Scapy Library:*** *Install via terminal: pip install scapy*
- ***Npcap/Libpcap:*** *Windows users need Npcap; Linux users need Libpcap (usually pre-installed).*

Execution

Because sniffing interacts directly with the Network Interface Card (NIC), it requires elevated privileges:

- ***Windows:*** *Open PowerShell/CMD as* ***Administrator*** *and run python sniffer.py.*
- ***Linux/macOS:*** *Run sudo python3 sniffer.py.*

6. What I Learned

Through this project, I gained practical experience in:

- ***The OSI Model:*** *Specifically Layers 3 (Network) and 4 (Transport).*
- ***Socket Programming:*** *Understanding how Python interacts with hardware-level network buffers.*

- ***Data Serialization:*** *Learning how raw bytes are transformed into human-readable headers.*
- ***Cybersecurity Ethics:*** *Understanding the legal boundaries and responsibilities associated with network monitoring tools.*

7. Conclusion

This Network Sniffer is a powerful entry-point into the world of network security and protocol analysis. While it effectively captures and categorizes traffic, it also highlights the critical importance of encryption (like TLS/SSL), as unencrypted data is easily visible to such tools. Future iterations of this project could include graphical traffic dashboards and automated threat detection for "suspicious" IP signatures.